

- Services hébergés sur ma VM apps et leur configuration
 - Introduction
 - Contexte professionnel
 - Technologies utilisées
 - Architecture du réseau
 - Tableau des services
 - Tableau des ports (VM apps)
 - Configuration des Équipements
 - 1. Corriger les chemins en dur après le transfert
 - 2. Résoudre le crash-loop d'intelligence-hub
 - 3. Nettoyer la base de données de n-core-api
 - 4. Gérer le conflit Telegram de hp-bot
 - 5. Mettre en place Traefik avec des certificats HTTPS mkcert
 - 6. Configurer la redirection HTTPS et le routage TLS
 - 7. Ajouter les nouveaux services au fil de l'eau
 - Tests et Résultats
 - Difficultés Rencontrées et Solutions
 - Améliorations Possibles
 - Ce que ce Projet Démontre
 - Références et Ressources

Services hébergés sur ma VM apps et leur configuration

Détail des services que j'héberge sur ma VM Proxmox « apps » (Docker), leur configuration, le reverse proxy Traefik avec HTTPS local, et les bugs rencontrés en les redéployant depuis mon ancien serveur.

Auteur : Garlens Charles

Formation : BTS SIO – Option SISR

Année : 2025-2026

Introduction

J'héberge une douzaine de services sur ma VM Docker « apps » (192.168.2.20), la plupart migrés depuis mon ancien serveur physique. Ce document explique comment chaque service est configuré, comment j'ai mis en place un reverse proxy Traefik avec de vrais certificats HTTPS locaux (mkcert), et surtout tout ce qui a cassé pendant que je faisais la migration — parce que c'est là que j'ai le plus appris.

Contexte professionnel

- **Continuité de service** : redéployer un service sur une nouvelle machine sans casser sa config (chemins, variables d'environnement) est une compétence directement transférable à n'importe quelle migration d'infra en entreprise.
- **Diagnostic par les logs** : le crash-loop d'intelligence-hub m'a appris à ne pas redéployer à l'aveugle, mais à lire

```
docker logs
```

pour comprendre la vraie cause avant d'agir.
- **Exposition sécurisée** : Traefik + HTTPS, même en interne, c'est le même principe qu'un reverse proxy en production — centraliser le point d'entrée, chiffrer, ne pas exposer les ports bruts.
- **Gestion des secrets** : les mots de passe générés (Pi-hole, Gitea, Postgres) sont dans des fichiers dédiés hors dépôt Git, jamais en clair dans un compose versionné.
- **Contraintes tierces** : comprendre qu'une API externe (Telegram long-polling) impose une contrainte d'architecture (une seule instance active) fait partie du travail réel d'un admin sys/réseau.

Technologies utilisées

- Docker

+

docker compose

— exécution des services applicatifs sur la VM apps

- Traefik 3.2.1

— reverse proxy, binaire officiel épinglé, service systemd dédié

- mkcert

— génération de certificats HTTPS locaux valides, sans avertissement navigateur

- PostgreSQL 16

— base de données partagée (LXC db, 192.168.2.13)

- Gitea 1.22.3

— hébergement Git (LXC git, 192.168.2.12)

- Pi-hole

— DNS et filtrage (LXC dns, 192.168.2.11)

Architecture du réseau

Tous mes services applicatifs tournent sur une seule VM (apps, 192.168.2.20) en conteneurs Docker, chacun sur son port. Traefik, sur un LXC séparé (edge, 192.168.2.10), fait office de point d'entrée unique : il reçoit le trafic sur 80/443 et le redirige vers le bon port de la VM apps selon le nom de domaine demandé.

Tableau des services

Équipement	Adresse IP	Rôle
apps (VM 120)	192.168.2.20	Hôte Docker de tous les services applicatifs
edge (CT 110)	192.168.2.10	Reverse proxy Traefik, certificats HTTPS
db (CT 113)	192.168.2.13	PostgreSQL partagé
git (CT 112)	192.168.2.12	Gitea
dns (CT 111)	192.168.2.11	Pi-hole

Tableau des ports (VM apps)

Ressource	Valeur
portfolio	3001
doc-forge	5000
cv-app	8080
kb-webhook	5555
intelligence-hub + newsletter-api	4000
ncore-api	3100
adminer	8081
Vaultwarden	8082
Uptime Kuma	3002
Homepage	3003
hub-dashboard	7777

Configuration des Équipements

1. Corriger les chemins en dur après le transfert

Après avoir copié doc-forge et kb-webhook sur la VM apps, leurs volumes Docker pointaient encore vers

```
/home/gls
```

(mon ancien utilisateur sur le HP), qui n'existe pas sous le nouvel utilisateur

```
debian
```

Pourquoi cette étape est importante ? Sans ce fix, les conteneurs démarrent mais avec des volumes vides ou en erreur — le service tourne en apparence mais ne sert

pas les bonnes données. C'est le genre de bug silencieux qu'on ne voit qu'en testant vraiment le contenu, pas juste

```
docker ps
```

.

```
grep -rL "/home/gls" ~/doc-forge/docker-compose.yml ~/kb-webhook/docker-  
compose.yml  
sed -i 's#/home/gls#/home/debian#g' ~/doc-forge/docker-compose.yml  
sed -i 's#/home/gls#/home/debian#g' ~/kb-webhook/docker-compose.yml  
sed -i 's#/home/gls#/home/debian#g' ~/scripts/.watchdog.env  
mkdir -p /home/debian/knowledge-base /home/debian/livrables  
scp -r gls@100.121.13.37:~/knowledge-base/* /home/debian/knowledge-base/
```

```
intelligence-hub
```

avait en plus son port bindé en dur sur l'ancienne IP du HP :

```
sed -i 's#100.121.13.37:4000:80#4000:80#' ~/intelligence-hub/docker-  
compose.yml
```

Résultat : les 3 services démarrent sans erreur de chemin, volumes correctement montés.

2. Résoudre le crash-loop d'intelligence-hub

```
docker ps
```

montrait

```
intelligence-hub
```

en

```
Restarting
```

en boucle juste après la migration.

Pourquoi cette étape est importante ? C'est ma première vraie séance de diagnostic Docker en conditions réelles : sans lire les logs, j'aurais pu perdre du temps à recréer le conteneur en boucle sans jamais comprendre que le problème venait d'un autre service.

```
docker logs --tail 50 intelligence-hub
# → nginx: [emerg] host not found in upstream "newsletter"

docker compose -f ~/intelligence-hub/docker-compose.yml ps newsletter
# → Exited (0) 4 days ago

docker compose -f ~/intelligence-hub/docker-compose.yml up -d newsletter
docker restart intelligence-hub

docker logs --tail 20 intelligence-hub
# → plus d'erreur upstream

curl -I http://localhost:4000
# → HTTP/1.1 200 OK
```

Résultat :

```
intelligence-hub
```

stable après redémarrage de sa dépendance

```
newsletter-api
```

3. Nettoyer la base de données de n-core-api

Le

```
docker-compose.yml
```

de n-core-api embarquait son propre conteneur Postgres local, redondant avec la vraie base déjà migrée sur le LXC

```
db
```

Pourquoi cette étape est importante ? Faire tourner deux bases différentes pour le même service (une locale au conteneur, une centralisée) aurait cassé la cohérence des données dès que quelqu'un écrirait dans la mauvaise. Autant le corriger tout de suite plutôt que de le découvrir en production.

```
# suppression du bloc "postgres:" entier dans docker-compose.yml
nano ~/n-core-api/docker-compose.yml

cat > ~/n-core-api/.env <<EOF
DB_HOST=192.168.2.13
DB_PORT=5432
DB_NAME=ncore
DB_USER=ncore
DB_PASSWORD=$(cat ~/draken-infra/ansible/.secrets/db_ncore_password.txt)
EOF

docker compose -f ~/n-core-api/docker-compose.yml up -d --build
curl http://localhost:3100/health
```

Résultat :

```
ncore-api
```

répond

```
healthy
```

, connecté à la vraie base partagée du LXC

```
db
```

.

4. Gérer le conflit Telegram de hp-bot

```
hp-bot
```

fait du long-polling Telegram (

```
getUpdates
```

) : une seule instance peut être active à la fois avec le même token.

Pourquoi cette étape est importante ? Je ne voulais pas couper le bot de l'ancien HP avant d'avoir validé toute la nouvelle infra pendant 48h. Sans cette précaution, j'aurais pu me retrouver avec deux instances en conflit (

```
409 Conflict
```

) et perdre les deux pendant la transition.


```
docker compose -f ~/hp-bot/docker-compose.yml build
docker compose -f ~/hp-bot/docker-compose.yml up -d
docker stop hp-bot
docker update --restart=no hp-bot
```

Résultat : le conteneur est prêt (image buildée, testée) mais arrêté volontairement, pour ne basculer qu'au moment choisi.

5. Mettre en place Traefik avec des certificats HTTPS mkcert

Pourquoi cette étape est importante ? Sans certificat valide, chaque accès à mes services en interne affichait un avertissement de sécurité dans Chrome — gênant au quotidien, et pas très pro à montrer si je fais une démo.

```
mkcert -install
mkcert "portfolio.gls" "gitea.gls" "pihole.home" "hub-dash.gls" "forge.gls" "
    cv.gls" "news.gls" "ncore.gls"

scp portfolio.gls+7.pem root@192.168.2.10:/etc/traefik/certs/cert.pem
scp portfolio.gls+7-key.pem root@192.168.2.10:/etc/traefik/certs/key.pem
ssh root@192.168.2.10 "chown -R traefik:traefik /etc/traefik/certs && \
    chmod 644 /etc/traefik/certs/cert.pem && \
    chmod 600 /etc/traefik/certs/key.pem"
```

Résultat : certificats en place — mais Traefik continuait d'afficher son certificat par défaut au premier essai (voir difficulté n°3 ci-dessous).

6. Configurer la redirection HTTPS et le routage TLS

Pourquoi cette étape est importante ? Avoir le certificat ne suffit pas : il faut explicitement dire à Traefik quel routeur utilise quel certificat, et forcer la redirection HTTP → HTTPS pour qu'un visiteur qui tape l'URL sans

```
https://
```

finisse quand même chiffré.

```
/etc/traefik/traefik.yml
```

:

```
entryPoints:
  web:
    address: ":80"
    http:
      redirections:
        entryPoint:
          to: websecure
          scheme: https
  websecure:
    address: ":443"
```

```
/etc/traefik/dynamic/routes.yml
```

(extrait) :

```
tls:
  certificates:
    - certFile: "/etc/traefik/certs/cert.pem"
      keyFile: "/etc/traefik/certs/key.pem"

http:
  routers:
    portfolio:
      rule: "Host(`portfolio.gls`)"
      entryPoints: ["websecure"]
      tls: {}
      service: portfolio-svc
  services:
    portfolio-svc:
      loadBalancer:
        servers:
          - url: "http://192.168.2.20:3001"
```

```
systemctl reload traefik
```

Résultat :

```
portfolio.gls
```

répond en HTTPS avec un vrai cadenas vert, plus d'avertissement Chrome.

7. Ajouter les nouveaux services au fil de l'eau

Pourquoi cette étape est importante ? Une infra qui évolue doit rester facile à étendre : à chaque nouveau service, je voulais que le processus soit le même trois lignes de commande, pas une nouvelle réflexion à chaque fois.

Pour Vaultwarden, Uptime Kuma et Homepage : même déploiement Docker sur

```
apps
```

, nouveau bloc router+service dans

```
routes.yml
```

, régénération du certificat mkcert avec le nouveau domaine inclus.

```
mkcert "portfolio.gls" "gitea.gls" "pihole.home" "hub-dash.gls" "forge.gls" \
      "cv.gls" "news.gls" "ncore.gls" "vault.gls" "uptime.gls" "home.gls"
```

Résultat :

```
vault.gls
```

,

```
uptime.gls
```

,

```
home.gls
```

accessibles en HTTPS, comme les services précédents.

Tests et Résultats

Test	Commande	Résultat
État des conteneurs sur apps	<code>docker ps</code>	 tous les services attendus "Up"
HTTP portfolio	<code>curl -I http://localhost:3001</code>	 200
HTTP doc-forge	<code>curl -I http://localhost:5000</code>	 200
HTTP cv-app	<code>curl -I http://localhost:8080</code>	 200
HTTP intelligence-hub	<code>curl -I http://localhost:4000</code>	 200 (après fix newsletter) 
Santé ncore-api	<code>curl http://localhost:3100/health</code>	connecté à la base ncore
Réponse kb-webhook	<code>curl -I http://localhost:5555</code>	 up (404 sur <code>/</code>  , normal, pas de route racine)
HTTPS via Traefik	Chrome sur https://portfolio.gls	  cadenas vert
Cert lu par Traefik	<code>openssl s_client -connect 192.168.2.10:443 -servername portfolio.gls</code>	puis après correction des permissions

- Le test

```
docker ps
```

confirme que le redéploiement n'a pas juste créé les conteneurs, mais qu'ils restent up dans la durée (pas de crash-loop résiduel).

- Les tests HTTP confirment que chaque service répond réellement, pas seulement que son process est lancé.
- Le test HTTPS/certificat confirme que Traefik sert bien mon certificat mkcert, pas son certificat auto-généré.

Difficultés Rencontrées et Solutions

#	Problème	Cause	Solution
1	Volumes Docker pointant vers /home/gls , absent sur la nouvelle VM	Chemins en dur dans les docker-compose.yml , jamais paramétrés	sed sur chaque compose concerné + recopie des dossiers manquants
2	intelligence-hub en crash-loop après migration	Upstream nginx (newsletter) arrêté depuis 4 jours, nginx refuse un upstream qui ne résout pas	docker compose up -d newsletter avant de relancer intelligence-hub
3	Traefik affichait son certificat par défaut au lieu du mien	Fichiers cert/clé appartenant à root après le scp , pas à l'utilisateur système traefik	chown traefik:traefik + permissions 644 / 600 explicites
4	Port d' intelligence-hub bindé en dur sur l'IP de l'ancien HP	100.121.13.37:4000:80 dans le compose, IP obsolète sur ce réseau	Bind standard 4000:80
5	Risque de conflit Telegram si hp-bot tournait des deux côtés	Long-polling Telegram n'autorise qu'une	Conteneur buildé et prêt mais volontairement stoppé (

#	Problème	Cause	Solution
		instance active par token	--restart=no)

Améliorations Possibles

Sécurité renforcée - Ajouter une authentification devant le dashboard

Traefik

(actuellement

insecure: true

, accessible sans mot de passe en interne) - Passer PostgreSQL de

md5

à

scram-sha-256

pour l'authentification - Stocker les mots de passe générés (Pi-hole, Gitea, Postgres) dans

Vaultwarden

plutôt que dans de simples fichiers texte sur le bastion

Fiabilité - Ajouter un

healthcheck

Docker sur chaque service pour que

docker ps

reflète vraiment l'état applicatif, pas juste « le process tourne » - Déclarer un ordre de démarrage explicite (

depends_on

+

condition: service_healthy

) entre

newsletter

et

intelligence-hub

, et entre

db

et

ncore-api

Supervision - Désactiver la boucle de monitoring passive de

hp-bot

une fois

Uptime Kuma

bien en place, pour éviter la double alerte sur le même incident - Ajouter des alertes Telegram différenciées par service critique plutôt qu'un statut global unique

Infrastructure - Terminer la bascule DNS des clients vers

Pi-hole

(actuellement testée via fichier hosts local, pas encore la vraie bascule DHCP) -
Ajouter les routes Traefik manquantes si l'accès par nom devient nécessaire pour
d'autres services internes

Ce que ce Projet Démontre

Compétence	Description
Migration applicative	Redéploiement de plus de 10 services Docker sur une nouvelle infrastructure, correction de configuration un par un
Diagnostic	Identification d'un crash-loop lié à une dépendance entre conteneurs, par lecture de logs plutôt que redéploiement à l'aveugle
Reverse proxy et TLS	Mise en place de Traefik avec certificats HTTPS locaux (mkcert), gestion fine des permissions de fichiers sensibles
Rigueur opérationnelle	Vérification systématique par test HTTP après chaque déploiement
Gestion des contraintes tierces	Compréhension d'une limite d'API externe (long-polling Telegram) et adaptation du plan de bascule

Références et Ressources

- Documentation officielle Traefik (routers, services, TLS)
- Documentation officielle mkcert (github.com/FiloSottile/mkcert)
- Documentation officielle Docker Compose
- Documentation officielle PostgreSQL (authentification,

```
pg_hba.conf
```

```
)
```

- Documentation Bot API Telegram (long polling,

```
getUpdates
```

```
)
```